Fundamentals of Programming Languages

Exercises 4

Untyped Lambda-calculus

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

The exercise numbers in this list refer to the exercise numbers in book Types and Programming Languages. Benjamin Pierce, The MIT Press, 2002.

Exercises in TAPL

1. Define the logical functions `or` and `not` (exercise 5.2.1).
2. Define a term for raising one number to the power of another (exercise 5.2.4).
3. Give a careful proof that $|\text{FV}(t)| \leq \text{size}(t)$ for every term t (exercise 5.3.3). Start by adapting the definition of size in chapter 3 to lambda terms.
4. Adapt the call-by-value rules of figure 5-3 to describe the other three strategies for evaluation: full beta-reduction, normal-order, and lazy evaluation (exercise 5.3.6).
5. Formulate the call-by-value evaluation rules in big-step style (exercise 5.3.8).

Implementation The following exercises build on the Haskell source code adapted from TAPL and provided at the course website.

6. Study the data type `Term` in module `UntypedLambda`.
7. Complete function `freeVars` in the module, which computes the set of free variables in a term (function `subs` depends on it). Refer to Definition 5.3.2. Suggestion: use module `Data.Set`.
8. Write a function that computes the `size` of a term. Adapt the definition of chapter 3.
9. Write a function that computes the `depth` of a term. Adapt the definition of chapter 3.

10. Write `Terms` for `id`, `true`, `false`, `test`, `and`, `or` and `not`.

11. Write `Terms` for `pair`, `fst`, and `snd`.

12. Write `Terms` for the first few numerals, `c0`, `c1`, ..., as well as operators `succ`, `plus`, `times` and `power`.

13. Write a predicate to determine whether a given term is a value.

14. Write the `step` function that performs one step of call-by-value evaluation. Use function `subs` from the module.

15. Write a function `eval` that repeatedly calls function `step`, until it finds a value.

16. Make sure that the following terms evaluate as in the textbook, where `v` and `w` are values of your choice:

    - `id (id (λz.id z))`;
    - `test true v w`;
    - `and true true`;
    - `and true false`;
    - `fst (pair v w)`.

17. Use `eval` to compute `succ c0`. Is the resulting term syntactically equal to `c1`? Discuss.

18. Write a function `bigStep` that implements the big-step evaluation rules of item 5.

19. Compare function `bigStep` against function `eval` by running the examples in item 16. Since `Term` does not derive `Eq`, compare the results by inspecting their `show` output.

Vasco T. Vasconcelos
October 10, 2026